Repetition (loops)

CSE 101 Section 03

Introduction to Algorithms and Computing

Fall 2023

© Royal Military College of Canada

Repetition

Display your name 5 times
Display your name 50 times
Display your name 5000002000 times

Repeat the code?

- Bulky code
- Mistakes
- Changes needed in many places
- Monotonous
- Hard to read

2 Types of Repetition Structures

Clap 6 times

- Count controlled
- "For" loop – "*for some number of times do:*"

Clap until I turn around

- Condition Controlled
- "While" loop – "*while this condition is true do:*"

Condition-controlled Loops

The `while` Loop: a Condition-Controlled Loop

while condition is true, do something

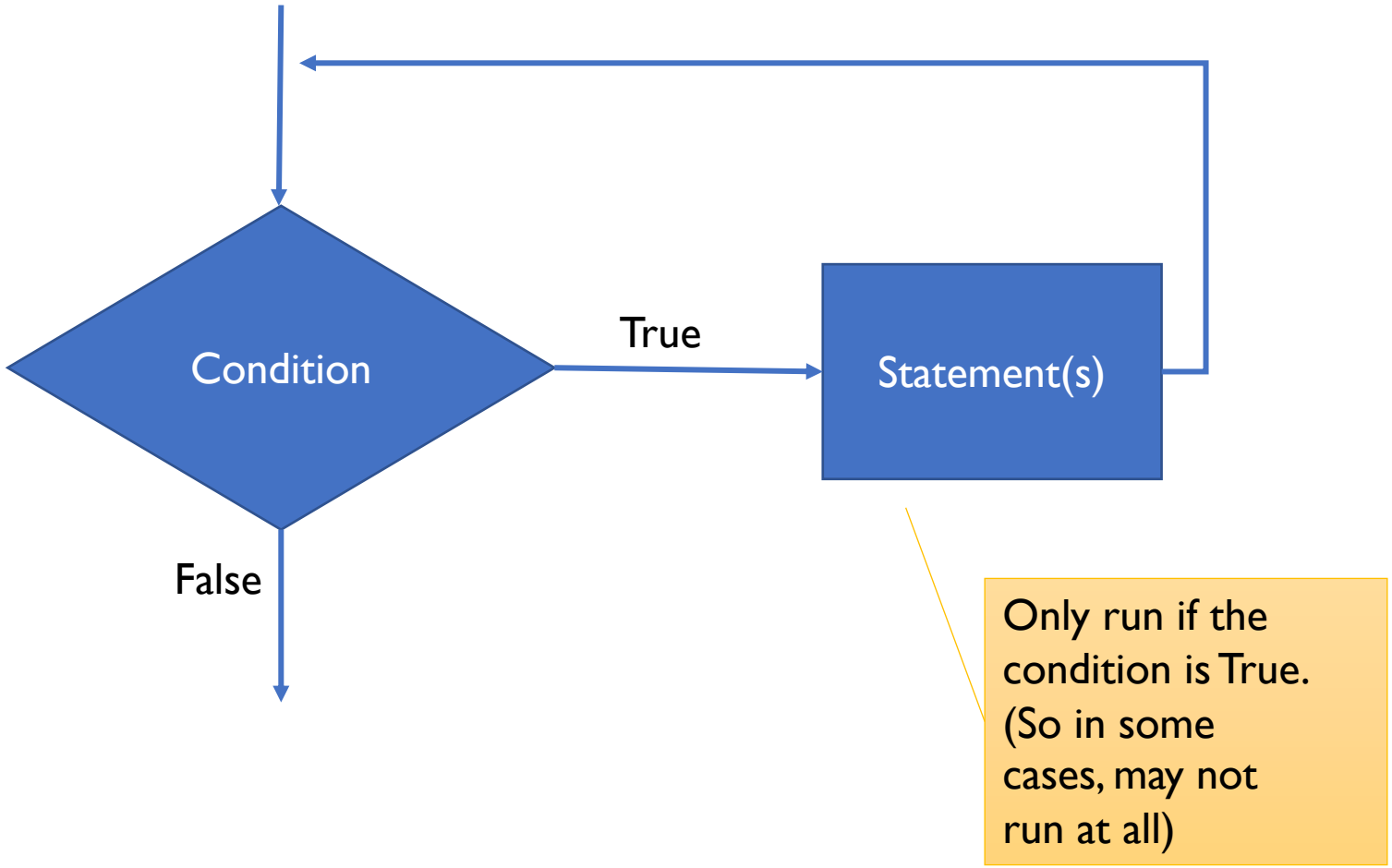
- Two parts:
  - Condition tested for true or false value
  - Statements repeated as long as condition is true

- General format:

```
while condition:
    statement
    statement
    statement

program continues
```

The `while` Loop



The `while` Loop

In order for a loop to stop executing, something has to happen inside the loop to make the condition false.

```
choice = 3
while choice < 5:
    print("Enter 5 to quit")
    # Get user input and convert to an integer
    choice = int(input("new choice? "))

# The loop will stop when the user enters a choice
of 5 or greater.
```

The `while` Loop

If there is no change in state to make the loop stop, we have an infinite loop.

It will run technically forever.

```
choice = 3
while choice < 5:
    print("Enter 5 to quit")
```

The `while` Loop

- Iteration: one execution of the body of a loop

- `while` loop is known as a *pretest* loop
  - Tests condition before performing an iteration
    - Will never execute if condition is false to start with
    - Requires performing some steps prior to the loop

Example `while` Loop

Count backwards from 30 by 7's until you reach zero or a negative number

```
currentNum = 30
while currentNum > 0:
    print(currentNum)
    currentNum = currentNum - 7

print("Finished!!")
```

Count-controlled Loops

Count-Controlled Loops

- Do 30 push-ups
- Clap your hands 12 times
- Print the happy birthday song 15 times

The `for` Loop: a Count-Controlled Loop

iterates a specific number of times

General format:

```
for variable in [val1, val2, etc]:
    # Code to be executed for each repetition
```

- Target variable: the variable which is the target of the assignment at the beginning of each iteration

- You should NOT modify the target variable inside the loop – this is done automatically!

The `for` Loop

- 1st iteration

```
for num in [1, 2, 3, 4, 5]:
    print(num)
```

- 2nd iteration

```
for num in [1, 2, 3, 4, 5]:
    print(num)
```

- 3rd iteration

```
for num in [1, 2, 3, 4, 5]:
    print(num)
```

- 4th iteration

```
for num in [1, 2, 3, 4, 5]:
    print(num)
```

- 5th iteration

```
for num in [1, 2, 3, 4, 5]:
    print(num)
```

range()

- The **range** function simplifies the process of writing a for loop
  - `range` returns an <u>iterable</u> object
    - <u>Iterable</u>: contains a sequence of values that can be iterated over

```
for i in range(3):
    print(i)
```

```
for i in [0, 1, 2]:
    print(i)
```

`range()`

- **`range` characteristics: If given:**
  - <u>One</u> argument: it will be used as ending limit
  - <u>Two</u> arguments: starting value and ending limit
  - <u>Three</u> arguments: third argument is step value

```
range(4)          ➔ [0 , 1 , 2 , 3]
range(5 , 8 )     ➔ [5 , 6 , 7]
range(2 , 10 , 2 ) ➔ [2 , 4 , 6 , 8]
```

Note

`range(endValue)`

- The `endValue` indicates "UP TO AND NOT INCLUDING" this value
- `range(2)` does NOT include 2.

More uses of `range()`

- **The `range` function can be used to generate a sequence with numbers in <u>descending</u> order**
  - Make sure starting number is larger than end limit, and step value is negative
  - Example: `range (10, 0, -1)`

Count backwards by 7's

- Remember:

```
currentNum = 30
while currentNum > 0:
    print(currentNum)
    currentNum = currentNum - 7
```

- Using a `for` loop:

```
for i in range(30 , 1 , -7):
    print(i)
```

Using the Target Variable Inside the Loop

- Purpose of target variable is to reference each item in a sequence as the loop iterates.

- While we <u>do not change/update</u> the target variable, it <u>can be used</u> in calculations or tasks in the body of the loop.

- Let's look at an example: calculate square of each number in a range.

Calculate square of each number in a range

```
for i in range(10):
    print(i , "squared is" , i ** 2)
```

- NOTE – we are NOT changing the value of `i`, just using the value.

Nested Loops

- Nested loop: loop that is <u>contained inside</u> another loop

```
for i in range(20):
    for j in range(5):
        do something
```

Nested Loops (Cont'd.)

Key point about nested loops:

- Inner loop goes through all of its iterations for each single iteration of outer loop.

Nested Loops: example

```
for i in range(1,3):
    print("i is ", i)
    for j in range(1 , 10):
        print(i , "*", j, "=", i*j)
    print("------------")
```

```
i is  1
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5
1 * 6 = 6
1 * 7 = 7
1 * 8 = 8
1 * 9 = 9
------------
i is  2
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
------------
```

For vs. While Loops

- **All for loops can be written as while loops**
  - The reverse is not true!

Next…

Practice Exercises